Documento de Diseño (D2) · Capítulo de arquitectura y despliegue

CascoVision · Sistema de detección automatizada de EPP mediante visión computacional

Iahn Vera · Joaquín Armijo · Benjamín Cáceres · Fernando Muñoz

Capstone PTY4614 · 2026.02 · Versión 1.0 del capítulo (tarea 3.1) · 20-10-2026

Índice

Control del documento 2
1. Introducción 2
1.1 Propósito 2
1.2 Entradas 2
1.3 Vistas que contiene 2
2. Decisiones de arquitectura 3
3. Vista lógica: componentes 5
3.1 Componentes del sistema 5
3.2 Módulos del agente Edge 6
3.3 Organización interna de la API 7
4. Comunicación entre servicios 8
5. Flujo principal y presupuesto de latencia 9
5.1 Detección de una persona sin EPP (CU-02 y CU-04) 9
5.2 Presupuesto de latencia (RNF-02) 10
6. Vista de despliegue 11
6.1 Nodos 11
6.2 Contenedores 12
6.3 Despliegue de desarrollo y pruebas 12
7. Atributos de calidad 13
7.1 Seguridad y privacidad 13
7.2 Disponibilidad y tolerancia a fallas 13
7.3 Escalabilidad 14
8. Trazabilidad de requisitos no funcionales 14
9. Estado de implementación y brechas 15
10. Puntos a confirmar en la revisión del equipo 15

Control del documento

Versión	Fecha	Autor	Descripción
0.7	15-10-2026	Equipo CascoVision	Arquitectura resumida en el README del repositorio y en la presentación de la semana 10
1.0	20-10-2026	Iahn Vera	Capítulo completo: decisiones de arquitectura, componentes, comunicación entre servicios, presupuesto de latencia, diagrama de despliegue, atributos de calidad y brechas con el código actual

Responsable: Iahn Vera (tarea 3.1). **Apoyo:** Joaquín Armijo. **Metodología:** tradicional en cascada, fase 3 (Diseño).

Este capítulo forma parte del **Documento de Diseño (D2)**. Se integra con el modelo de datos (tarea 3.2), el protocolo de la baliza (3.3), los mockups (3.4), los diagramas UML (3.5) y el diseño Docker (3.6) en la consolidación del D2 (tarea 3.8). Su aprobación forma parte del hito «D2 y plan de pruebas aprobados».

1. Introducción

1.1 Propósito

Este capítulo describe **cómo se organiza CascoVision**: qué componentes tiene, cómo se comunican, dónde se ejecuta cada uno y por qué se tomaron esas decisiones. Es la referencia para construir los componentes (fase 4), para integrarlos (fase 5) y para el manual de despliegue (D4).

1.2 Entradas

- **SRS v0.4** (D1, 20-10-2026): 19 requisitos funcionales, 18 no funcionales y 11 casos de uso. En particular, las decisiones cerradas en su §4.6 (autenticación, reintentos de la baliza, evidencia y salidas de la baliza).
- **Código del repositorio** al 20-10-2026 (rama dev-iahn): API FastAPI (src/api/), migraciones de PostgreSQL (db/migrations/), motor de detección y agente Edge provisional (Yolo/testing y modelamiento/) y contrato de la API (docs/API.md).
- **Decisiones del equipo:** el agente Edge emite la alerta física (24-09-2026); la baliza es un M5Stack StickS3 con sonido y pantalla, sin vibración (23-09-2026); el despliegue usa Docker Compose (23-09-2026).

1.3 Vistas que contiene

Vista	Pregunta que responde	Sección

Vista	Pregunta que responde	Sección
Decisiones de arquitectura	¿Por qué el sistema tiene esta forma?	§2
Vista lógica	¿Qué componentes hay y qué hace cada uno?	§3
Comunicación entre servicios	¿Qué se envían, por qué protocolo y con qué autenticación?	§4
Flujo principal	¿Cómo se cumple la alerta en menos de 5 s?	§5
Vista de despliegue	¿En qué equipos y contenedores corre cada componente?	§6
Atributos de calidad	¿Cómo se cumplen seguridad, disponibilidad y escalabilidad?	§7

2. Decisiones de arquitectura

El estilo general es una **arquitectura Edge on-premise**: todo el procesamiento ocurre en un servidor dentro de la faena, y solo salen a Internet las notificaciones de texto.

ID	Decisión	Motivo	Alternativa descartada	Requisitos
AD-01	Todo el software corre en un **servidor Edge dentro de la red de la faena**. El video nunca sale de la obra.	Privacidad (Ley N° 19.628) y menor latencia. No depende de la conexión a Internet.	Procesar en la nube (AWS o Azure), descartado en el Informe 3.	RNF-08, RNF-09, §4.4 del SRS
AD-02	El **agente Edge envía la orden de alerta directamente a la baliza**, sin pasar por la API.	Ahorra un salto de red, protege el límite de 5 s y la alerta funciona aunque la API o la base de datos estén caídas.	La API como emisora de la alerta (versión 0.1 del SRS).	RF-07, RNF-02
AD-03	La alerta va **antes** que el difuminado, el guardado de la evidencia y el registro en la API.	Solo la detección, la evaluación de zona y la orden a la baliza están en la ruta crítica.	Registrar primero y alertar después.	RNF-02, CU-02
AD-04	El agente informa a la API de forma **asíncrona**, con una **cola en disco**.	Una API lenta o caída no debe detener el análisis de las cámaras ni las alertas siguientes.	Envío síncrono dentro del ciclo de análisis (versión actual, ver §9).	RNF-13, CU-02 (7a)
AD-05	La comunicación entre componentes usa **HTTP/REST con JSON**, y el dashboard recibe los cambios por **WebSocket**.	Es simple, la conoce todo el equipo y FastAPI la soporta de forma nativa. Alcanza para el volumen de un MVP.	MQTT con un broker Mosquitto: queda como evolución si crece el número de balizas.	RF-13, RNF-04
AD-06	La **evidencia se guarda en un volumen del servidor Edge**. La base de datos guarda	No se guardan imágenes en la base de datos ni se exponen rutas del	Guardar la imagen en PostgreSQL (BYTEA) o servir la carpeta como	RF-05, RF-06, RF-19

ID	Decisión	Motivo	Alternativa descartada	Requisitos
	solo la ruta relativa, y la API entrega la imagen solo a usuarios autenticados.	disco.	archivos estáticos públicos.	
AD-07	**Un único punto de entrada:** el nginx del dashboard publica el puerto 443 y reenvía /api y /ws a la API. La API y la base de datos no publican puertos.	Reduce la superficie expuesta y permite un solo certificado HTTPS.	Publicar la API y la base de datos en la LAN.	RNF-07, RNF-10
AD-08	La **inferencia corre en CPU** en el servidor Edge. El entrenamiento se hace con GPU en Google Colab.	El cliente no necesita comprar una GPU. A 1 cuadro por segundo por cámara, la CPU alcanza para el MVP (a medir en la tarea 4.3).	GPU dedicada en el servidor Edge.	RNF-01, §4.3 del SRS
AD-09	Todo se despliega con **Docker Compose en un solo equipo**. Un segundo agente Edge en otro equipo puede apuntar a la misma API.	Se instala con un solo comando y escala por configuración.	Kubernetes, desproporcionado para una faena.	RNF-12, RNF-15
AD-10	La **capa de almacenamiento de la API es intercambiable**: PostgreSQL en el despliegue y un archivo JSON solo para desarrollo y pruebas.	Permite probar la API sin levantar una base de datos. Ya está implementado (almacen.py).	Depender siempre de PostgreSQL.	RNF-17

3. Vista lógica: componentes

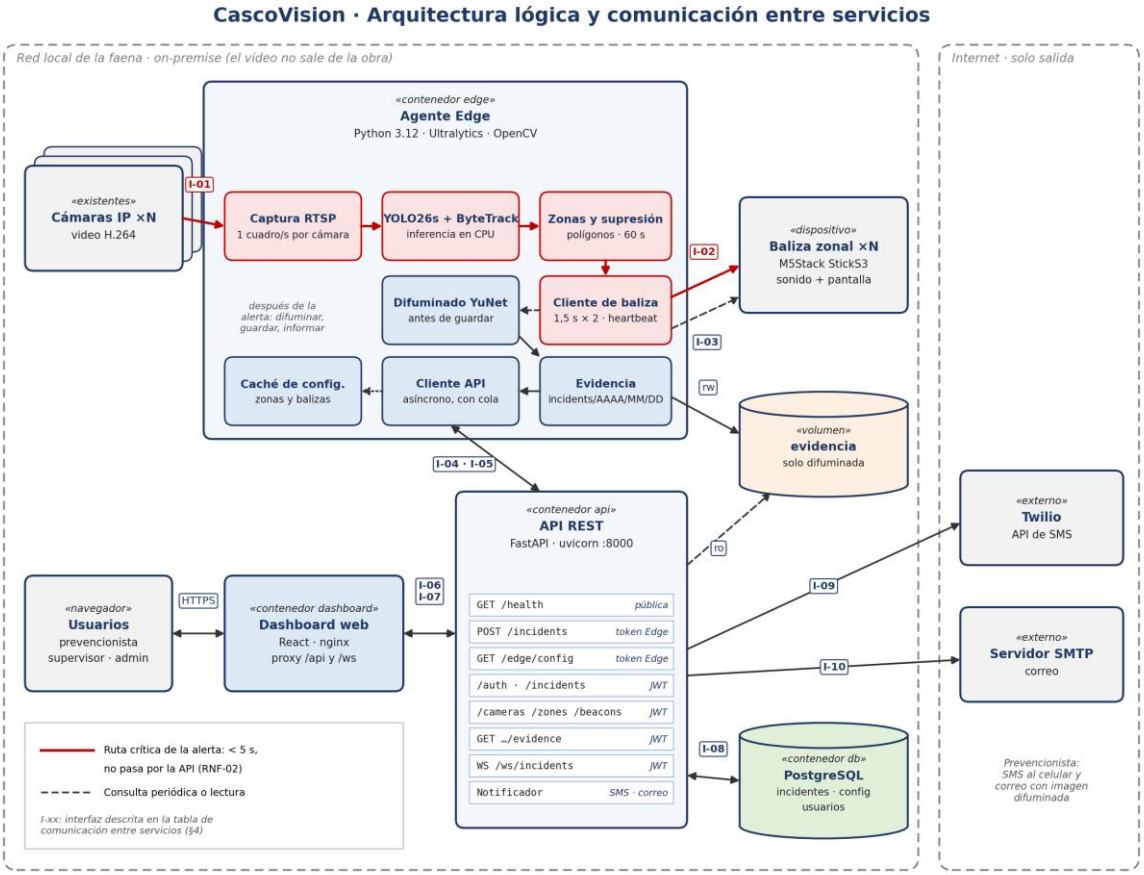


Figura 1. Arquitectura lógica de CascoVision y comunicación entre servicios. La línea roja es la ruta crítica de la alerta.

3.1 Componentes del sistema

Componente	Responsabilidad	Tecnología	Contenedor	Requisitos
Cámaras IP	Entregan el video. Son las que la faena ya tiene.	RTSP, H.264	No aplica (existentes)	RF-01
Agente Edge	Analiza el video, decide si hay infracción dentro de una zona, activa la baliza, difumina los rostros, guarda la evidencia e informa a la API.	Python 3.12, Ultralytics (YOLO26s, ByteTrack), OpenCV (YuNet)	edge	RF-01 a RF-08, RF-18
Baliza zonal	Recibe la orden de alerta, suena y muestra en pantalla la zona y el EPP faltante. Responde al *heartbeat*.	M5Stack StickS3 (ESP32-S3), firmware con M5Unified	No aplica (dispositivo)	RF-07, RF-09, RF-10, RNF-14
API REST	Registra incidentes, guarda la configuración, autentica	FastAPI, uvicorn,	api	RF-04, RF-05, RF-09 a RF-

Componente	Responsabilidad	Tecnología	Contenedor	Requisitos
	usuarios, entrega la evidencia, publica eventos en tiempo real y envía las notificaciones.	psycopg 3		19
Base de datos	Persiste incidentes, cámaras, zonas, balizas, usuarios y roles.	PostgreSQL	db	RF-05, RF-15, RF-16
Dashboard web	Monitoreo en tiempo real, gestión de incidentes, reportes y configuración.	React, servido con nginx	dashboard	RF-13 a RF-16, RNF-16
Twilio y servidor SMTP	Entregan el SMS y el correo al prevencionista.	Servicios externos	No aplica	RF-11, RF-12

3.2 Módulos del agente Edge

El agente Edge es el componente más crítico, porque concentra la ruta de la alerta. Se divide en módulos con una responsabilidad cada uno, para poder probarlos por separado (tarea 4.22).

Módulo	Función	En la ruta crítica	Estado al 20-10
Captura RTSP	Abre el flujo de cada cámara, toma 1 cuadro por segundo y descarta el resto. Si la cámara se cae, reconecta con espera creciente.	Sí	Pendiente: hoy lee una cámara local por índice y analiza todos los cuadros
Detector y seguimiento	YOLO26s detecta personas, casco y chaleco; ByteTrack asigna a cada persona un número anónimo que se mantiene entre cuadros.	Sí	Implementado (motor_cascovision.py)
Zonas y supresión	Verifica si la persona está dentro de un polígono de riesgo de esa cámara y suprime las alertas repetidas de la misma persona o zona durante 60 s.	Sí	Parcial: registra solo cuando cambia la infracción de una persona; faltan la ventana de 60 s y los polígonos
Cliente de baliza	Resuelve la IP de la baliza de la zona, envía POST /alert (1,5 s por intento y un reintento) y consulta GET /health de cada baliza cada 30 s.	Sí	Pendiente (tareas 4.4 y 4.6)
Difuminado	Difumina los rostros con YuNet, con respaldo por la posición de la cabeza. El cuadro original nunca se guarda.	No	Implementado

Módulo	Función	En la ruta crítica	Estado al 20-10
Evidencia	Guarda la imagen difuminada en incidents/AAAA/MM/DD/ y devuelve la ruta relativa.	No	Implementado
Cliente API	Envía el incidente a la API desde un hilo aparte. Si la API no responde, lo deja en una cola en disco y lo reenvía cuando vuelve.	No	Implementado de forma síncrona (brecha B-01, §9)
Caché de configuración	Obtiene de la API las cámaras, zonas y balizas, las guarda en disco y las renueva cada 60 s. Si la API está caída al arrancar, usa la última copia.	No	Pendiente (tareas 4.4 y 4.15)

3.3 Organización interna de la API

La API sigue el diseño que ya tiene el código: los modelos de entrada y salida se validan con Pydantic, la autenticación se resuelve con dependencias de FastAPI y la persistencia queda detrás de la interfaz Almacen (almacen.py), así las rutas no dependen de dónde se guardan los datos.

Grupo de rutas	Quién la usa	Autenticación	Estado
GET /health	Docker (*healthcheck*), monitoreo	Pública, sin datos	Implementada
POST /incidents	Agente Edge	Token del agente Edge (Bearer)	Implementada
GET /incidents, GET /incidents/{id}, PATCH /incidents/{id}	Dashboard	JWT	Consultas implementadas sin JWT; cambio de estado pendiente
GET /incidents/{id}/evidence	Dashboard	JWT (Prevencionista, Supervisor o Administrador)	Pendiente
GET /edge/config, PUT /edge/beacons/{id}/status	Agente Edge	Token del agente Edge	Pendiente
/auth/login, /users	Dashboard	Credenciales y JWT	Pendiente
/cameras, /zones, /beacons	Dashboard (administrador)	JWT con rol Administrador	Pendiente
/reports	Dashboard	JWT	Pendiente
WS /ws/incidents	Dashboard	JWT en el *handshake*	Pendiente
Notificador (tarea interna)	Se activa al registrar un	No aplica	Pendiente

Grupo de rutas	Quién la usa	Autenticación	Estado
	incidente		

4. Comunicación entre servicios

Cada interfaz tiene un identificador (I-xx) que aparece en la Figura 1. Las interfaces I-02 e I-03 se detallan en la especificación del protocolo de la baliza (tarea 3.3), y el contrato de I-04 está en docs/API.md.

ID	Origen → destino	Protocolo y puerto	Contenido	Autenticación	Frecuencia y espera
I-01	Cámara IP → agente Edge	RTSP sobre TCP, :554	Flujo de video H.264. El agente toma 1 cuadro por segundo	Usuario y clave de la cámara, en variables de entorno	Continua. Si se pierde, reconecta con espera creciente
I-02	Agente Edge → baliza	HTTP, :80, POST /alert	JSON con la zona, el tipo de infracción y la duración (1–30 s)	Token compartido de la baliza (BALIZA_TOKEN)	Por infracción. 1,5 s por intento y un reintento; sin respuesta a los 3 s, la alerta queda «fallida»
I-03	Agente Edge → baliza	HTTP, :80, GET /health	Estado, batería y tiempo encendida	Pública, sin datos sensibles	Cada 30 s. Tras 3 fallos seguidos, la baliza queda fuera de línea
I-04	Agente Edge → API	HTTP, :8000 (red interna), POST /incidents	Incidente en JSON: cámara, zona, tipos de infracción, confianza, número de seguimiento, ruta de la evidencia y resultado de la alerta	Authorization: Bearer con EDGE_API_TOKEN	Por infracción, asíncrono. Si falla, cola en disco
I-05	Agente Edge ↔ API	HTTP, :8000, GET /edge/config y PUT /edge/beacons/{id}/status	Configuración de cámaras, polígonos de zonas y balizas (IP, duración, sonido). Cambios de	Bearer con EDGE_API_TOKEN	Configuración al arrancar y cada 60 s. Estado de las balizas solo cuando cambia

ID	Origen → destino	Protocolo y puerto	Contenido	Autenticación	Frecuencia y espera
			estado de las balizas		
I-06	Dashboard → API	HTTPS, :443 (nginx) → :8000	Consultas y comandos REST en JSON	JWT con expiración de hasta 8 h	Por acción del usuario
I-07	API → dashboard	WebSocket seguro, :443 (nginx) → :8000	Incidentes nuevos y cambios de estado de balizas y cámaras	JWT en el *handshake*	Al ocurrir el evento (≤ 3 s, RNF-04)
I-08	API → PostgreSQL	TCP, :5432 (red interna)	Consultas SQL	Usuario y clave de PostgreSQL, en variables de entorno	Por petición
I-09	API → Twilio	HTTPS, :443 (salida a Internet)	Texto del SMS: zona, tipo de infracción y hora	Credenciales de Twilio, en variables de entorno	Por incidente, ≤ 30 s (RNF-03)
I-10	API → servidor SMTP	SMTP con STARTTLS, :587 (salida a Internet)	Resumen del incidente e imagen difuminada	Usuario y clave SMTP, en variables de entorno	Por incidente, ≤ 30 s (RNF-03)

Además de estas interfaces, el agente Edge y la API comparten el **volumen de evidencia**: el agente lo escribe y la API lo monta en **solo lectura** para entregar las imágenes (RF-19). Así la imagen nunca viaja por la red interna dentro del JSON del incidente.

5. Flujo principal y presupuesto de latencia

5.1 Detección de una persona sin EPP (CU-02 y CU-04)

1. El agente Edge toma un cuadro de la cámara (I-01).
2. YOLO26s detecta las personas y su EPP, y ByteTrack les asigna un número de seguimiento anónimo.
3. El agente verifica que la persona esté dentro de una zona de riesgo y que no exista una alerta vigente de la misma persona o zona (supresión de 60 s).
4. **El agente envía POST /alert a la baliza de la zona (I-02).** La baliza suena y muestra la zona y el EPP faltante. Fin de la ruta crítica.
5. El agente difumina los rostros y guarda la evidencia en el volumen.
6. El agente informa el incidente y el resultado de la alerta a la API (I-04), sin esperar la respuesta dentro del ciclo de análisis.

7. La API registra el incidente en PostgreSQL (I-08), lo publica al dashboard (I-07) y envía el SMS y el correo (I-09 e I-10) como tarea en segundo plano.

El diagrama de secuencia UML de este flujo forma parte de la tarea 3.5.

5.2 Presupuesto de latencia (RNF-02)

Etapa	Presupuesto	Cómo se cumple
Espera del cuadro (muestreo a 1 FPS)	≤ 1,0 s	El peor caso es que la infracción empiece justo después de un cuadro
Decodificación, inferencia y seguimiento	≤ 0,5 s	Inferencia en CPU con YOLO26s. Se mide en la tarea 4.3; si no alcanza, se exporta a ONNX o se baja la resolución de entrada
Zonas y supresión	< 0,01 s	Operaciones geométricas en memoria
Orden a la baliza	≤ 3,0 s	1,5 s por intento y un reintento. En una LAN sana responde en menos de 0,1 s
Total en el peor caso	**≤ 4,5 s**	**Bajo el límite de 5 s**

Las etapas 5 a 7 del flujo están fuera de la ruta crítica: no suman a la latencia de la alerta física. La notificación remota tiene su propio límite de 30 s (RNF-03).

6. Vista de despliegue

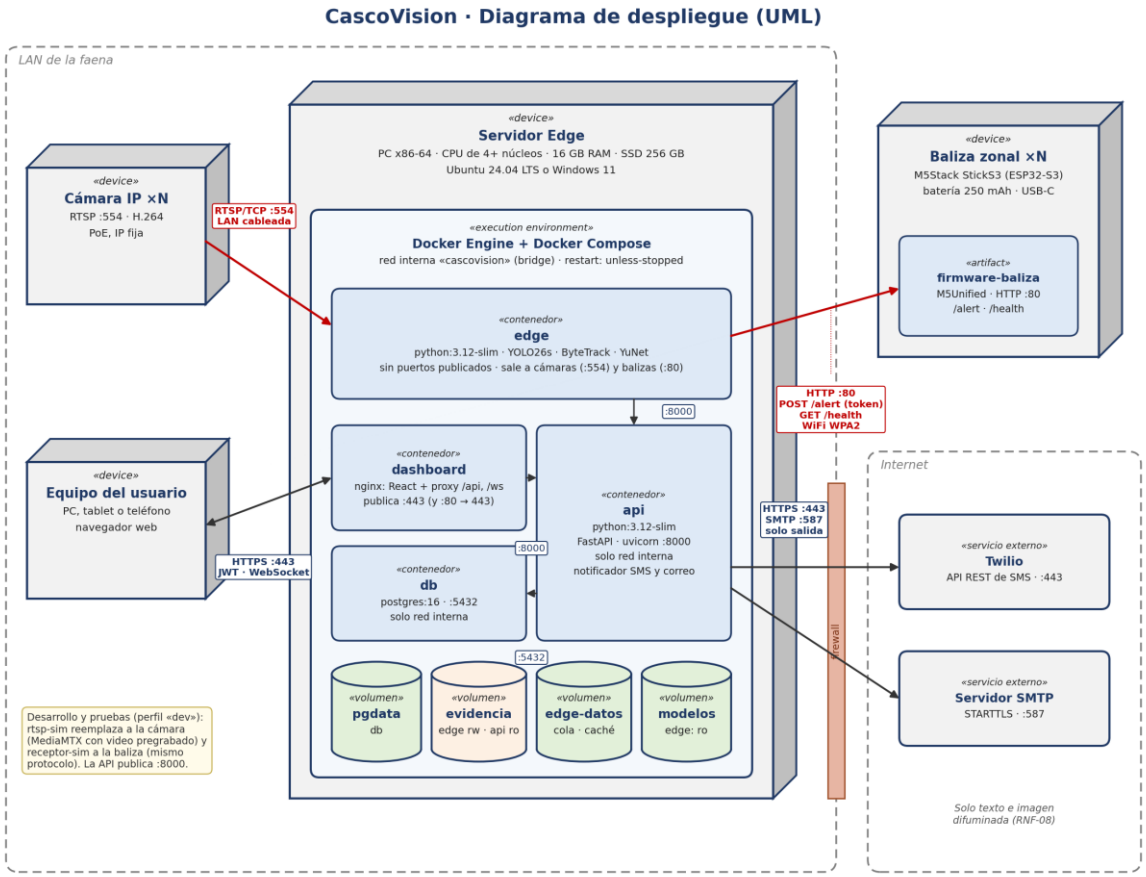


Figura 2. Diagrama de despliegue UML de CascoVision en la red de la faena.

6.1 Nodos

Nodo	Tipo	Características	Qué ejecuta
Servidor Edge	Equipo físico en la faena	Requisito mínimo propuesto: PC x86-64 con CPU de 4 o más núcleos, 16 GB de RAM y SSD de 256 GB; Ubuntu 24.04 LTS o Windows 11 con Docker	Los 4 contenedores del sistema y sus volúmenes
Cámara IP	Dispositivo existente	RTSP, H.264, alimentación PoE e IP fija	Su propio firmware
Baliza zonal	Dispositivo IoT, una por zona	M5Stack StickS3 (ESP32-S3), WiFi de 2,4 GHz, batería de 250 mAh	firmware-baliza (M5Unified, servidor HTTP en el puerto 80)
Equipo del usuario	PC, tablet o teléfono	Navegador web actualizado	El dashboard, cargado desde el servidor Edge
Twilio y servidor SMTP	Servicios externos en Internet	Solo reciben conexiones salientes desde la API	Envío de SMS y correo

6.2 Contenedores

Esta tabla fija lo que el despliegue necesita de cada contenedor. Las imágenes, los Dockerfiles, las redes y las variables de entorno se detallan en el diseño Docker (tarea 3.6) y se construyen en la tarea 4.17.

Contenedor	Imagen base	Puertos	Volúmenes	Depende de	Verificación de salud
db	postgres:16	5432, solo red interna	pgdata	No aplica	pg_isready
api	python:3.12-slim	8000, solo red interna	evidencia (solo lectura)	db en buen estado	GET /health
dashboard	nginx (React compilado en una etapa previa con Node)	443 publicado; 80 redirige a 443	Certificado HTTPS	api	Respuesta de nginx en /
edge	python:3.12-slim con Ultralytics y OpenCV	Ninguno publicado	evidencia (lectura y escritura), edge-datos (cola y caché), modelos (solo lectura)	No depende de api para arrancar: usa la configuración guardada	Archivo de latido que el agente actualiza cada ciclo

Todos los contenedores usan restart: unless-stopped, de modo que Docker los reinicia si fallan y los levanta al encender el servidor (RNF-13). Los secretos se cargan desde un archivo .env que no se sube al repositorio (RNF-10); el repositorio solo contiene .env.example.

6.3 Despliegue de desarrollo y pruebas

Para desarrollar y probar sin cámaras ni balizas reales, el docker-compose.yml tendrá un perfil dev con dos servicios más:

Servicio	Reemplaza a	Función
rtsp-sim	Cámara IP	Servidor RTSP (MediaMTX) que transmite en bucle un video pregrabado de la faena
receptor-sim	Baliza zonal	Servicio que implementa el mismo protocolo (POST /alert, GET /health) y registra cada orden recibida con su hora, para medir la latencia (tarea 4.6)

En este perfil la API publica además el puerto 8000, para usar la documentación interactiva (/docs) y las pruebas de integración. El agente Edge apunta al simulador o al equipo real solo cambiando variables de entorno, sin cambios de código.

7. Atributos de calidad

7.1 Seguridad y privacidad

La arquitectura separa tres zonas de confianza:

Zona	Qué contiene	Qué entra	Qué sale
Internet	Twilio y el servidor SMTP	Nada: no hay puertos abiertos hacia la faena	No aplica
LAN de la faena	Cámaras, balizas y equipos de los usuarios	Solo el puerto 443 del servidor Edge (dashboard)	Solo la API sale a Internet, por los puertos 443 y 587
Red interna de Docker	api, db, edge y dashboard	Solo el tráfico que reenvía nginx	No aplica

Controles que aplica la arquitectura:

- **Privacidad:** el video no sale del servidor Edge. El agente difumina los rostros antes de guardar, y las notificaciones envían solo texto y la imagen difuminada (RNF-08, RNF-09).
- **Autenticación por tipo de cliente:** token del agente Edge en POST /incidents (RNF-18), JWT de hasta 8 h para los usuarios y contraseñas con bcrypt (RNF-06), y token compartido en la orden a la baliza (RNF-11).
- **Control de acceso por rol** en cada ruta de la API (RNF-07). La evidencia se entrega solo a usuarios autenticados, y la ruta guardada se valida como relativa en la API y en la base de datos.
- **Secretos** solo en variables de entorno (RNF-10).
- **HTTPS** en el único puerto publicado, con un certificado propio de la faena.

Limitación conocida. La orden a la baliza viaja por HTTP sin cifrar, así que el token podría verse dentro de la LAN. Se mitiga poniendo las balizas y las cámaras en una red WiFi de dispositivos separada (WPA2) o en una VLAN propia. La evolución es MQTT con TLS. Del mismo modo, hoy todos los agentes Edge comparten un mismo token; con más de un agente conviene un token por agente.

7.2 Disponibilidad y tolerancia a fallas

Falla	Efecto	Comportamiento diseñado
La API o la base de datos se caen	No se registran incidentes en línea	La baliza sigue sonando, porque la orden no pasa por la API (AD-02). El agente guarda los incidentes en la cola en disco (hasta 500) y los reenvía cuando la API vuelve
Una baliza queda fuera de línea	No hay alerta física en esa zona	El *heartbeat* la detecta en 90 s como máximo. El agente no espera su respuesta y registra la alerta como «omitida». El incidente igual se registra y se notifica, y el dashboard muestra la baliza fuera de línea
Una cámara se desconecta	No se analiza ese sector	El agente reintenta la conexión con espera creciente, y el dashboard muestra la cámara sin

Falla	Efecto	Comportamiento diseñado
		transmisión (RF-18). Las demás cámaras siguen funcionando
Se cae Internet	No salen SMS ni correos	La API reintenta el envío. La alerta física, el registro y el dashboard no se ven afectados
Falla un contenedor	El servicio deja de responder	Docker lo reinicia (restart: unless-stopped) y su verificación de salud lo marca como no disponible mientras tanto
Se reinicia el servidor Edge	Todo el sistema se detiene unos minutos	Docker levanta los contenedores al arrancar. Los volúmenes conservan la base de datos, la evidencia y la cola. El agente arranca con la configuración guardada
Se pierde el disco del servidor	Pérdida de datos	Respaldo diario de la base de datos y la evidencia (tarea 4.13)

7.3 Escalabilidad

- **Cámaras:** un agente Edge procesa hasta 4 cámaras a 1 cuadro por segundo en CPU (RNF-01). Para más cámaras se agrega un segundo agente en otro equipo, que se conecta a la misma API (AD-09).
- **Zonas y balizas:** se agregan desde el dashboard, sin cambios de código (RNF-12). El agente recibe la nueva configuración en un máximo de 60 s.
- **Datos:** los incidentes se consultan por fecha, zona y cámara; el índice por fecha ya existe en la migración 002 y los demás se definen en el modelo de datos (tareas 3.2 y 4.9).

8. Trazabilidad de requisitos no funcionales

Requisito	Cómo lo cumple la arquitectura	Dónde se verifica
RNF-01 · 1 FPS por cámara, hasta 4 cámaras	Muestreo en el módulo de captura, inferencia en CPU (AD-08)	Prueba de rendimiento (5.4)
RNF-02 · Alerta < 5 s	Ruta crítica corta (AD-02, AD-03) y presupuesto de §5.2	Prueba de rendimiento con receptor-sim (5.4)
RNF-03 · Notificación ≤ 30 s	Notificador en segundo plano en la API (I-09, I-10)	Prueba de integración (5.3)
RNF-04 · Dashboard ≤ 3 s	WebSocket (I-07)	Prueba de integración (5.3)
RNF-06, RNF-07 · JWT y roles	Dependencias de autenticación en la API	Prueba de seguridad (5.5)
RNF-08, RNF-09 · Privacidad	AD-01, difuminado antes de guardar, AD-06	Revisión de la evidencia y prueba de seguridad (5.5)
RNF-10 · Secretos	Variables de entorno y .env fuera del repositorio	Revisión del repositorio (5.5)
RNF-11, RNF-18 · Tokens	Token de la baliza (I-02) y del agente Edge (I-04)	Pruebas de la API (ya existen) y del receptor
RNF-12 · Crecimiento por configuración	Configuración desde la API (I-05) y varios agentes (AD-09)	Prueba de integración con 2 balizas (5.2)
RNF-13 · Disponibilidad ≥ 99	Reinicio automático, cola en disco	Prueba de integración con la

Requisito	Cómo lo cumple la arquitectura	Dónde se verifica
%	y fallas de §7.2	API caída (5.3)
RNF-14 · *Heartbeat*	Cliente de baliza (I-03)	Prueba con receptor-sim apagado (5.3)
RNF-15 · Docker	Despliegue de §6	Instalación limpia con docker compose up (D4)

9. Estado de implementación y brechas

El código actual cubre la API de incidentes y el motor de detección. Estas son las diferencias entre ese código y el diseño, y la tarea de la planificación que las cierra:

ID	Brecha	Riesgo	Tarea
B-01	El agente provisional (monitoreo_api.py) envía el incidente a la API **dentro del ciclo de análisis**: si la API no responde, se detiene hasta 40 s por incidente (3 intentos de 10 s más 2 esperas de 5 s).	Mientras espera no analiza cuadros ni envía alertas, lo que rompe RNF-02 justo cuando la API falla.	4.4 (hilo de envío y cola, AD-04)
B-02	La captura lee una cámara local por índice y analiza todos los cuadros.	No se conecta a cámaras IP ni respeta el muestreo de 1 FPS.	4.4
B-03	La cámara y la zona están fijas en el código (camara-01, zona-norte) y no hay polígonos de zona.	Toda la imagen cuenta como zona de riesgo.	4.4 y 4.15 (I-05)
B-04	No existe el cliente de baliza: el agente informa todas las alertas como «omitidas» (skipped_offline).	No hay alerta física.	4.4, 4.6 y 4.7
B-05	Faltan en la API la autenticación JWT, la configuración, la entrega de evidencia, el WebSocket y el notificador.	Las consultas de incidentes están abiertas sin JWT.	4.15, 4.16, 4.19 y 4.20
B-06	No hay docker-compose.yml ni Dockerfiles, y el agente vive en Yolo/testing y modelamiento/ en vez de src/edge/.	El sistema no se puede levantar con un comando (RNF-15).	4.17 y 4.4
B-07	Las rutas de la evidencia y de la cola están fijas dentro del repositorio.	En Docker deben apuntar a los volúmenes evidencia y edge-datos.	4.4 (rutas por variables de entorno)

10. Puntos a confirmar en la revisión del equipo

1. **Responsable del *heartbeat*:** este diseño lo asigna al agente Edge, porque es el único componente que ya habla con las balizas y necesita saber si están en línea antes de alertar. La API solo recibe los cambios de estado (I-05).
2. **WebSocket para el dashboard** (I-07), en lugar de consultar la API cada pocos segundos.

3. **Requisito mínimo del servidor Edge** (§6.1), a confirmar con la medición de la tarea 4.3.
4. **Red separada para balizas y cámaras** (§7.1), y si la demostración usa un router propio.
5. **Tiempo de retención de la evidencia** en el volumen (se define con el modelo de datos, tarea 3.2).